



CMC UNIVERSITY

Aspire to Inspire the Digital World

Chương 4.

Tương tác với cơ sở dữ liệu bằng ORM (Entity Framework hoặc framework tương đương)

- 4.1. Thiết kế model và tạo migration
- 4.2. Kết nối và xử lý các thao tác cơ bản với dữ liệu gồm: select, insert, delete, update, filter,
- 4.3. Triển khai thao tác CRUD với mẫu thiết kế DAO (DAO pattern) và Repository.
- 4.4. Lập trình thao tác CRUD trong cơ chế bất đồng bộ
- 4.5. Thực hành với Web API và Web MVC .NET core hoặc framework tương đương kết nối và thao tác CRUD với CSDL bằng Entity hoặc framework tương đương

- ORM (Object-Relational Mapping) là một kỹ thuật ánh xạ dữ liệu từ các bảng trong cơ sở dữ liệu quan hệ (RDBMS) thành các đối tượng trong ngôn ngữ lập trình.
- Loại bỏ sự phức tạp của việc viết các câu lệnh SQL thủ công.

Giới thiệu ORM

- Ví dụ minh họa ánh xạ:

UserID	Username	Email
1	Alice	alice@email.com

- Lớp đối tượng trong lập trình

```
public class User
```

```
{
```

```
    public int UserID { get; set; }
```

```
    public string Username { get; set; }
```

```
    public string Email { get; set; }
```

```
}
```

- Năng suất cao hơn: Không cần viết thủ công SQL, chỉ tập trung vào logic nghiệp vụ.
- Tính di động: Dễ dàng thay đổi RDBMS (SQL Server, MySQL, PostgreSQL) mà không phải thay đổi nhiều mã nguồn.
- Giảm lỗi: Tự động hóa ánh xạ giúp hạn chế lỗi viết sai SQL

Nhược điểm của ORM

- Hiệu năng: Với các truy vấn phức tạp, ORM có thể kém tối ưu so với SQL thủ công.
- Độ phức tạp: Học và sử dụng ORM yêu cầu hiểu rõ về nguyên lý hoạt động.

Cấu trúc Entity Framework

- Database First: Bắt đầu từ cơ sở dữ liệu hiện có, tạo các lớp ánh xạ tự động.
- Code First: Xây dựng mô hình từ mã nguồn, tự động tạo và quản lý cơ sở dữ liệu.

Thêm gói NuGet:

- `Install-Package Microsoft.EntityFrameworkCore`
- `Install-Package Microsoft.EntityFrameworkCore.SqlServer`
- `Install-Package Microsoft.EntityFrameworkCore.Tools`

Cấu hình DbContext:

```
public class ApplicationDbContext : DbContext
{
    public
    ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
        : base(options) { }

    public DbSet<User> Users { get; set; }
}
```

Ánh xạ lớp với bảng:

```
[Table("Users")]  
public class User  
{  
    [Key]  
    public int UserID { get; set; }  
    [Required]  
    public string Username { get; set; }  
    public string Email { get; set; }  
}
```

- Sử dụng Migration để quản lý cơ sở dữ liệu:
- Tạo Migration:

Add-Migration InitialCreate

- Cập nhật cơ sở dữ liệu:

Update-Database

Thực hiện thao tác CRUD với EF

```
{  
    // Create  
    var newUser = new User { Username = "Alice", Email = "alice@email.com" };  
    context.Users.Add(newUser);  
    context.SaveChanges();  
  
    // Read  
    var user = context.Users.FirstOrDefault(u => u.Username == "Alice");  
  
    // Update  
    user.Email = "alice.new@email.com";  
    context.SaveChanges();  
  
    // Delete  
    context.Users.Remove(user);  
    context.SaveChanges();  
}
```

- **Cấu hình Hibernate ORM trong Spring Boot**

1. Cài đặt Maven Dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

2. Cấu hình ứng dụng:

- Trong file application.properties:

`spring.datasource.url=jdbc:h2:mem:testdb`

`spring.datasource.driverClassName=org.h2.Driver`

`spring.jpa.hibernate.ddl-auto=update`

`spring.jpa.show-sql=true`

3. Tạo Entity và Repository:

Entity:

`@Entity`

```
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String username;  
    private String email;  
    // Getters và Setters  
}
```

3. Tạo Entity và Repository:

- Repository:

```
using (var context = new
ApplicationDbContext(options))

public interface UserRepository extends
JpaRepository<User, Long> {

    List<User> findByUsername(String username);

}
```

4. Thực hiện CRUD:

```
@RestController
@RequestMapping("/users")
public class UserController {
    @Autowired
    private UserRepository userRepository;

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userRepository.save(user);
    }

    @GetMapping("/{id}")
    public Optional<User> getUser(@PathVariable Long id) {
        return userRepository.findById(id);
    }
}
```

Yêu cầu cơ bản:

- Sử dụng ORM để ánh xạ dữ liệu.
- Thực hiện đầy đủ các thao tác CRUD.
- Sử dụng giao diện đơn giản (CLI hoặc Web).

Tiêu chí đánh giá:

- Mức độ hoàn thiện của ứng dụng.
- Sử dụng ORM đúng chuẩn.
- Tính sáng tạo và khả năng trình bày

1. Quản lý sinh viên:

- ✓ Chức năng: Thêm, sửa, xóa, tìm kiếm sinh viên.
- ✓ Công nghệ: Entity Framework hoặc Spring Boot.

2. Quản lý sản phẩm và đơn hàng:

- Chức năng: Quản lý thông tin sản phẩm, đặt hàng, theo dõi trạng thái đơn hàng.

3. Ứng dụng quản lý thư viện:

- Chức năng: Quản lý sách, mượn trả, theo dõi tình trạng sách.



CMC UNIVERSITY

Aspire to Inspire the Digital World

THANK YOU